

ÇÖZÜLEBİLİR VE DENGELİ 3D PUZZLE SEVİYE ÜRETİM SİSTEMİ

BlockMerge3D Oyunu İçin Manuel Parça Kütüphanesinden
Zorluk Kontrollü Yapay Zeka Üretimine Teknik Rehber

Belgenin Kapsamı ve Amacı

BlockMerge3D oyununda, 3D voxel tabanlı hedef şekillerin ve tamamlayıcı parçaların mantıklı zorluk dereceleriyle, katmanlı yapı (layer mode) kısıtlarıyla ve %100 çözülebilirlik garantisiyle üretilmesini sağlayacak sistemi tanımlamaktır. Kolay, orta, zor ve uzman seviyelerin nasıl sayısallaştırılacağını, manuel parçaların sisteme nasıl entegre edileceğini ve çözücü doğrulamasını açıklar.

Uygulama Odağı: Unity / C# / 3D Grid & Voxel Tabanlı Parça Yerleştirme
Son Güncelleme: Temmuz 2026

Yönetici Özeti

Mevcut sistemin en büyük sorunu, 3D seviye alanı (hedef şekil) ile tamamlayıcı parçaların birbirinden bağımsız veya aşırı rastgele üretilmesidir. Bir parça geometrik olarak geçerli olsa bile hedef 3D alana uygun olmayabilir; uygun görünen parçalar da bir araya geldiğinde çözümsüz çıkmazlar oluşturabilir. BlockMerge3D gibi katmanlı (Y-layer) 3D voxel yapılarında bu sorun daha da kritiktir.

Önerilen sistemin ana kuralı şudur: Seviye önce çözülmüş halde oluşturulur (Solution-First Generation). Sistem, kütüphaneden parçaları seçer, bu parçaları hedef 3D ızgaraya geçerli biçimde yerleştirerek bir çözüm kurar, adımları kaydeder ve ardından parçaları oyuncuya karıştırılmış ve yönleri değiştirilmiş biçimde sunar. Böylece en az bir çözüm doğrudan üretim sırasında garanti edilir.

Yapay zekaya kontrolsüz biçimde "rastgele 3D seviye üret" görevi verilmemelidir. AI; parça kütüphanesi, zorluk profili, dikey katman sınırlamaları ve çözülebilirlik doğrulayıcı tarafından denetlenen bir seçim ve optimizasyon katmanı olarak kullanılmalıdır.

İçindekiler

1. Problem Tanımı ve Tasarım Varsayımları	Sayfa 3
2. Temel İlke: Çözümünden Seviyeye Üretim (3D Uyumlu)	Sayfa 4
3. Önerilen Sistem Mimarisi ve Modüller	Sayfa 5
4. Manuel Parça Kütüphanesinin Yapısı (3D Voxel)	Sayfa 6
5. Parça Özelliklerinin Otomatik Çıkarılması (3D)	Sayfa 7
6. Seviye Zorluk Derecelerinin Sayısallaştırılması	Sayfa 8
7. Seviye Üretim Algoritması ve Backtracking	Sayfa 9
8. Çözülebilirlik ve Benzersiz Çözüm Kontrolü	Sayfa 10
9. Yapay Zekaya Seviye ve Parça Öğretilmesi	Sayfa 11
10. Unity Veri Yapıları ve Kod İskeleti (BlockMerge3D Uyumlu)	Sayfa 12
11. Test, Telemetri ve Dengeleme Metotları	Sayfa 13
12. Uygulama Yol Haritası (Fazlar)	Sayfa 14
13. Hatalı Yaklaşımlar ve Koruma Kuralları	Sayfa 15
Ek A ve B: Zorluk Profilleri ve Seviye Kabul Kontrol Listesi	Sayfa 16

1. Problem Tanımı ve Tasarım Varsayımları

Bu rehber, BlockMerge3D oyununun mevcut 3D voxel birleştirme yapısını esas alır. Oyun alanı, kare hücrelerden (3D grid) oluşur; her tamamlayıcı parça (piece) bir veya daha fazla bağlı küpten meydana gelir. Oyuncu, bu parçaları sürükleyip döndürerek hedef şekli boşluksuz doldurmalıdır.

1.1. Mevcut Seviye Üretimindeki Sorunlar:

- * **Bağımsız Üretim:** Tamamlayıcı parçalar ile ana hedef şekil geometrisi birbirinden kopuk üretilmektedir.
- * **Çözülebilirlik Kontrolü Eksikliği:** Parçaların tek başına geçerli olması, birlikte bir çözüm oluşturdıkları anlamına gelmez. Gerçek çözücü (Solver) çalıştırılmadığı için çıkmaz yollar içeren seviyeler üretilmektedir.
- * **Katman Uyumsuzluğu:** 3D katmanlar (Layer) açıldığında görünülük ve yerleştirme sırası bağımlılıkları hesaba katılmamaktadır.
- * **Sığ Zorluk Kriterleri:** Zorluk sadece parça sayısı üzerinden ölçülmekte; dallanma faktörü veya gecikmeli hata riskleri ihmal edilmektedir.

1.2. Başarı Ölçütleri:

- * **Çözülebilirlik:** Yayına giren her seviyenin en az 1 adet doğrulanmış çözümü bulunmalıdır.
- * **Zorluk Tutarlılığı:** Aynı zorluk profilindeki seviyelerin ortalama çözüm süresi ve hata oranları benzer aralıkta olmalıdır.
- * **Belirlilik (Seed):** Aynı Seed ve aynı zorluk profili her zaman birebir aynı seviyeyi üretmelidir.
- * **Performans:** Mobil cihazlarda bile doğrulama ve üretim süreleri milisaniyeler seviyesinde kalmalıdır.

2. Temel İlke: Çözümünden Seviyeye Üretim (3D Uyumlu)

En güvenli ve garanti seviye üretim yöntemi "Solution-First Generation" (Çözümünden Seviyeye Üretim) yaklaşımıdır. Bu yöntemde seviye, oyuncuya sunulan başlangıç (boş/karışık) durumundan değil, tamamlanmış 3D doğru yerleşimden türetilir.

2.1. Üretim Akış Adımları:

- * **Adım 1: İstenen zorluk profili (Kolay, Orta, Zor, Uzman) seçilir.**
- * **Adım 2: Profil kısıtlarına uygun hedef 3D boyutlar (X, Y, Z) ve hedef hücre sayısı belirlenir.**
- * **Adım 3: Manuel parça kütüphanesinden veya AI parça jeneratöründen aday parçalar seçilir.**
- * **Adım 4: Seçilen parçalar hedef 3D ızgaraya (boş board) geçerli backtracking mantığıyla yerleştirilir ve çözülmüş hedef şekil oluşturulur.**
- * **Adım 5: Bu yerleşimlerin dönüşleri, konumları ve dikey katman (Y) sıraları "Çözüm İzi" (Solution Trace) olarak kaydedilir.**
- * **Adım 6: Parçalar oyuncuya sunulmadan önce yönleri ve sıraları karıştırılır.**
- * **Adım 7: Bağımsız LevelSolver, oyundaki kurallarla seviyeyi çözerek çözülebilirliği doğrular.**

Neden bu yöntem? Çözüm izini üretim esnasında kaydetmek, seviyenin çözülebilirliğini teorik olarak garanti altına alır. Bağımsız çözücü ise sonradan yapılan karıştırma, dikey katman kısıtları veya engel (prefilled) yerleşimlerinin bu çözümü bozmadığını kesinleştirir.

3. Önerilen Sistem Mimarisi

BlockMerge3D seviye üretim sistemi, tek bir büyük sınıf yerine sorumlulukları ayrılmış modüllerden oluşmalıdır. Bu yapı AI'nın kontrolsüz veya hatalı seviyeler üretmesini engeller.

Modül Adı	Görevi	Çıktısı
Piece Library	Manuel ve yapay zeka parçalarını kategorize edip saklar.	PieceDefinition Listesi
Piece Analyzer	Voxel hacmi, 3D bounding box ve simetri özelliklerini hesaplar.	Geometrik Özellik Vektörü
Difficulty Profile	Zorluk derecesine göre kısıtları ve limitleri belirler.	Sayısal Kısıt Kümesi
Piece Selector	Profil kısıtlarına uygun parça setini seçer.	Aday Parça Seti
Solution Builder	Parçaları 3D ızgaraya yerleştirerek çözülmüş şekil kurar.	3D Grid + Çözüm İzi
Scrambler	Parça başlangıç konumlarını, yönlerini ve sıralarını karıştırır.	Oyuncu Başlangıç Durumu
Solver / Validator	Oyuncu kurallarıyla (katman katman) çözüm arar.	Geçerli/Çözümsüz, Çözüm Sayısı
Difficulty Evaluator	Seviye metriklerini çıkararak zorluk skoru üretir.	0-100 Zorluk Skoru
AI Policy Layer	Telemetriye göre seçim ağırlıklarını ve profilleri yönetir.	Optimize Ağırlık Dağılımı

Veri Akış Şeması:

Zorluk Profili -> Parça Seçici -> Çözüm Kurucu (3D Backtracking) -> Karıştırıcı -> Bağımsız LevelSolver Doğrulaması -> Zorluk Skoru Kontrolü -> Kayıt (LevelData Asset)

4. Manuel Parça Kütüphanesinin Yapısı (3D Voxel)

Manuel parça oluşturma editörü (PieceDesignerWindow), sadece şekli kaydeden bir araç olmamalıdır. Her 3D voxel parça, seviye üreticinin karar verebilmesi için makine tarafından analiz edilebilir veri yapısıyla (ScriptableObject) saklanmalıdır.

4.1. Voxel Parça Veri Yapısı (PieceDefinition):

- * **id:** Benzersiz parça kimliği (örn. P_3D_Corner_05)
- * **cells:** Pivot noktasına göre local 3D koordinatlar (Vector3Int türünde)
- * **allowedRotations:** Hangi 3D eksenlerde dönmeye izin verildiği (Euler 0, 90, 180, 270)
- * **volume (area):** Parçadaki toplam küp sayısı
- * **difficultyTags:** Tasarımcı etiketleri (örn. sharpEdge, flatBridge)
- * **spawnWeight:** Seviye jeneratöründe seçilme olasılığı çarpanı
- * **maxCopiesPerLevel:** Aynı seviyede bu parçanın en fazla kaç kez kullanılabileceği

4.2. 3D Hücre Koordinatlarının Normalleştirilmesi:

Aynı şeklin farklı konumlarda kaydedilmiş kopyalarını ayırt etmek ve kütüphanede mükerrer veri birikmesini önlemek için hücreler normalleştirilmelidir. Tüm koordinat kümesinden minimum X, minimum Y ve minimum Z çıkarılır; koordinatlar sıralanır ve benzersiz bir kanonik imza metni oluşturulur. Bu imza, parçanın kimlik kartıdır.

Örnek Koordinatlar: (2, 3, 1), (3, 3, 1), (2, 4, 1), (2, 3, 2)

Minimum Değerler: minX = 2, minY = 3, minZ = 1

Normalize Edilmiş: (0, 0, 0), (1, 0, 0), (0, 1, 0), (0, 0, 1)

Kanonik İmza: 0:0:0|0:0:1|0:1:0|1:0:0

5. Parça Özelliklerinin Otomatik Çıkarılması (3D)

Herhangi bir tasarımcı yeni bir voxel parça eklediğinde, sistem otomatik olarak şekli analiz edip sayısal niteliklerini (Features) hesaplar. Bu nitelikler AI parça seçicisine seviye zorluğunu tahmin etmede girdi sağlar.

Metrik / Özellik	Nasıl Hesaplanır / Anlamı	Seviye Zorluğuna Etkisi
Küp Sayısı (Hacim)	Toplam küp adedi (Volume).	Büyük parçalar yerleşim alanını daraltır.
Bounding Box	X, Y, Z eksenlerindeki maksimum boyutlar.	Geniş parçalar dikey katman sınırlarını zorlar.
Doluluk Oranı	Hacim / (En x Boy x Yükseklik).	Düşük doluluk, şeklin düzensiz ve zor olduğunu gösterir.
Yüzey Alanı	Dışarıya bakan açık küp yüzlerinin toplamı.	Yüksek yüzey alanı karmaşık geçişler yaratır.
Kompaktlık (3D)	Hacim / Yüzey Alanı oranı.	Düşük kompaktlığa sahip parçalar zor yerleşir.
3D Simetri Derecesi	Döndürüldüğünde aynı kalan yönlerin sayısı.	Yüksek simetri, yerleşim olasılığını artırır.
Dikey Yayılım	Y eksenindeki kalınlık / katman sayısı.	Dikey parçalar katmanlar arası bağımlılık kurar.
Uç Nokta Sayısı	Yalnızca tek bir komşu küpü olan hücreler.	İnce uzantısı çok olan parçalar hassastır.

5.1. Voxel Parça Ön Zorluk Puanı Formülü:

$$\text{pieceDifficulty} = 0.25 * (1 - \text{compactness3D}) + 0.20 * (1 - \text{symmetryScore3D}) + 0.20 * \text{densityDeviation} + 0.20 * \text{uniqueRotationCountScore} + 0.15 * \text{verticalSpreadingScore}$$

Sonuç 0 ile 100 arasına normalize edilir. Yüksek puan, zor yerleştirilen karmaşık parçaları gösterir.

6. Seviye Zorluk Derecelerinin Sayısallaştırılması

Kolay, Orta, Zor ve Uzman etiketleri soyut tanımlardan kurtarılıp ölçülebilir formüllere bağlanmalıdır. BlockMerge3D oyununda seviye zorluğunu belirleyen temel 3D kriterler aşağıdadır.

6.1. Zorluk Belirleyen Temel 3D Metrikler:

- * **Parça Adedi:** Seviyede birleştirilmesi gereken toplam parça sayısı.
- * **Dikey Katman Geçişleri (Y-Complexity):** Çözüm esnasında katmanlar arası dikey yerleşim sıklığı. Katmanlar arası geçiş zorunluluğu arttıkça hafıza yükü katlanır.
- * **Dallanma Faktörü (Branching Factor):** Çözücünün her adımda yapabileceği ortalama geçerli hamle sayısı. Düşük dallanma faktörü, hamle serbestliğini kısıtlar ve zorluğu artırır.
- * **Gecikmeli Hata Oranı (Delayed Failure):** İlk anda doğru görünen ancak ileriki adımlarda çıkmaza yol açan yanlış yerleşimlerin sıklığı.
- * **Sıra Bağımlılığı (Order Dependency):** Parçaların dikey katman veya engel blokları (prefilled/ice) nedeniyle belirli bir sırayla yerleştirilme zorunluluğu.

6.2. Önerilen 3D Seviye Zorluk Puanı Formülü:

$$\text{levelDifficulty} = 0.20 * \text{pieceCountScore} + 0.18 * \text{Y_LayerComplexity} + 0.18 * \text{delayedFailureRatio} + 0.16 * \text{orderDependencyScore} + 0.14 * \text{branchingFactorScore} + 0.14 * \text{pieceSimilarityRatio}$$

Zorluk Aralıkları:

- * 0 - 24 : Kolay Seviye
- * 25 - 49 : Orta Seviye
- * 50 - 74 : Zor Seviye
- * 75 - 100: Uzman Seviye

7. Seviye Üretim Algoritması

BlockMerge3D jeneratörü, rastgele voxel çizmek yerine hedefe yönelik kısıtlı backtracking algoritması kullanır. Üretim süreci aşağıdaki 4 aşamadan geçer:

Aşama 1: Hedef Alan Maskesi ve Profil Seçimi

Hedeflenen zorluk derecesine göre toplam hacim (küp adedi) ve 3D sınır kutusu (grid size) belirlenir. Kolay seviyeler için dar ve girintili 3D alanlar yerine daha kompakt yapılar tercih edilir.

Aşama 2: 3D Uyumlu Parça Seçimi

Parça seçici, parça kütüphanesinden toplam alanları hedef hacme eşit olacak şekilde aday parçalar belirler. Bu seçimde parçaların zorluk puanı dağılımı da profil kısıtlarına uymalıdır (örn. Kolay seviyede yüksek dikey yayımlı veya delik riski yüksek parçalar elenir).

Aşama 3: Çözülmüş 3D Şeklin Kurulması (Solution Builder)

Seçilen parça seti backtracking arama algoritmasıyla hedef 3D alana yerleştirilir. Arama sırası optimizasyonu için "en az yerleşim seçeneği olan büyük/düzensiz parçalar önce yerleştirilir" kuralı uygulanır. Eğer yerleşim sırasında doldurulamayacak izole boşluklar kalırsa arama dalı erken kesilir.

Aşama 4: Başlangıç Karıştırması (Scrambler)

Yerleşen parçalar hedeften çıkarılır, dikey başlangıç dönüş açıları (0/90/180/270 derece) ve oyuncuya sunulma sırası karıştırılarak başlangıç durumuna dönüştürülür.

8. Çözülebilirlik ve Benzersiz Çözüm Kontrolü

Solution Builder tarafından kaydedilen çözüm izi teorik bir garanti sunsa da, oyun içi katman geçişleri, sürüklenme engelleri ve benzeri kısıtlar nedeniyle seviyeyi test eden bağımsız bir Solver (LevelSolver.cs) çalıştırılmalıdır.

8.1. 3D Geri İzlemeli Çözücü Algoritması:

Çözücü, oyuncunun yapabileceği tüm hamleleri (döndürme ve 3D yerleştirme) simüle eder. En önemli performans optimizasyonu, her adımda en az olasılığa sahip parçayı seçip denemektir.

8.2. Çözüm Sayısının Sınırlandırılması ve Zorluğa Etkisi:

- * **Kolay Seviyeler:** Çoklu çözüm yolları kabul edilebilir; oyuncuya serbestlik tanınır.
- * **Zor Seviyeler:** Sadece tek bir (Unique) çözüm yolu bulunması hedeflenir. Alternatif yolların olmaması seviyeyi daha katı ve zihinsel olarak zorlayıcı yapar.

8.3. Arama Ağacını Erken Budama Kuralları:

- * **Hacim Eşitliği:** Kalan boş hücre sayısı, kalan parça hacimlerinin toplamına eşit olmalıdır.
- * **İzole Hacim Analizi:** Oluşan herhangi bir izole boş 3D bölgenin hacmi, kalan parça hacimlerinin alt kombinasyonları ile kapatılamıyorsa arama hemen iptal edilir.
- * **Katman Blokaj Kontrolü:** Üst katmanda yerleştirilecek bir parça, alt katmandaki boşlukları tamamen kapatıp erişilmez kılıyorsa yerleşim reddedilir.

9. Yapay Zekaya Nasıl Öğretilmeli?

Öğretme süreci doğrudan karmaşık derin öğrenme modelleri kurmayı gerektirmez. İlk aşamada, tasarımcıların manuel oluşturduğu parçalar ve seviyeler yapılandırılmış veri seti (JSON) haline getirilir. AI bu veri setinden kurallar, parça kombinasyonları ve zorluk çıkarımları öğrenir.

9.1. Manuel Seviye ve Parça Örneklerinden Öğrenme:

Oluşturulan her başarılı el yapımı seviye, sadece görsel olarak değil metadata nitelikleriyle birlikte kaydedilir. Bu veriler AI eğitim verisini (Training Dataset) oluşturur:

Eğitim Verisi JSON Yapısı Örneği:

```
{
  "levelId": "L_BM3D_HARD_104",
  "difficultyLabel": "Zor",
  "boardSize": {"x": 5, "y": 2, "z": 5},
  "pieceIds": ["P_Corner_3D_A", "P_Line_3D_B", "P_Flat_T_C"],
  "solution": [
    {"pieceId": "P_Corner_3D_A", "pos": {"x": 0, "y": 0, "z": 0}, "rot": 90},
    {"pieceId": "P_Line_3D_B", "pos": {"x": 2, "y": 1, "z": 0}, "rot": 180}
  ],
  "metrics": {
    "layerComplexity": 0.65,
    "delayedFailureRatio": 0.42,
    "solutionCount": 1,
    "difficultyScore": 72
  }
}
```

AI Katmanının Görevi: Oyuncu telemetrisinden (çözüm süreleri, pes etme oranları) yararlanarak zorluk skorlarını kalibre etmek ve parça seçici ağırlıklarını güncellemektedir. AI, Solver doğrulaması olmadan seviyeyi asla "çözülebilir" olarak onaylayamaz.

10. Unity Veri Yapıları ve Kod İskeleti

BlockMerge3D projesindeki ScriptableObject yapıları ile tam uyumlu veri sınıfları ve üretim kod iskeleti aşağıda tanımlanmıştır.

10.1. CubeShapeData (Hedef Şekil ve Hücre Tanımı):

```
using UnityEngine;
using System.Collections.Generic;

[CreateAssetMenu(fileName = "NewCubeShapeData", menuName = "BlockMerge3D/Cube Shape Data")]
public class CubeShapeData : ScriptableObject
{
    public string shapeName;
    public Vector3Int gridSize;
    public float cellSize = 1.0f;
    public float spacing = 0.1f;
    public List<Vector3Int> occupiedCells = new List<Vector3Int>();
    public List<Vector3Int> prefilledCells = new List<Vector3Int>();
    public List<int> prefilledMaterialIndices = new List<int>();
    public GameObject cubePrefab;
}
```

10.2. LevelData (Seviye Tanımı ve Kazanma Koşulları):

```
[CreateAssetMenu(fileName = "NewLevel", menuName = "BlockMerge3D/Level Data")]
public class LevelData : ScriptableObject
{
    public string levelName;
    public GameObject mainShapePrefab; // CubeShapeDataHolder içeren hedef prefab
    public List<GameObject> complementaryPieces = new List<GameObject>();
    public float timeLimit = 60f;
    public int targetScore = 100;
}
```

11. Test, Telemetri ve Dengeleme

Otomatik hesaplanan zorluk metrikleri sadece birer tahmindir. Gerçek zorluk derecesi, seviyeleri oynayan gerçek oyuncuların davranış verilerine (Telemetri) göre optimize edilmelidir.

11.1. Toplanması Gereken Temel Telemetri Olayları:

- * **level_start:** Seviyenin kaç kez başlatıldığını kaydeder.
- * **piece_placed:** Hangi parçanın hangi sırayla ve hangi Y katmanına yerleştirildiğini izler.
- * **undo:** Oyuncunun hamleyi geri alma sıklığı (yanlış yola girme oranı).
- * **restart / fail:** Pes etme veya sürenin bitmesi durumlarını raporlar.
- * **level_complete:** Toplam çözüm süresini ve harcanan hamle sayısını kaydeder.

11.2. Gerçek Zorluk Doğrulama Kriterleri:

Profil	Medyan Çözüm Süresi	Yeniden Başlatma Oranı	İpucu Kullanım Oranı
Kolay	15 - 45 saniye	%0 - %8	%0 - %5
Orta	40 - 90 saniye	%5 - %18	%3 - %12
Zor	80 - 180 saniye	%15 - %35	%10 - %25
Uzman	150+ saniye	%25 - %55	%20 - %45

12. Uygulama Yol Haritası

Üretim sistemini tek bir aşamada kurmak yerine, adım adım test edilebilir fazlara bölmek projenin sağlığı açısından kritik öneme sahiptir.

Geliştirme Fazları:

- * **Faz 1 - Parça Altyapısı:** PieceDefinition veri yapısı, 3D normalleştirme ve kanonik imza kontrol sisteminin kurulması. Manuel çizilen parçaların doğrulanması.
- * **Faz 2 - Deterministik Çözücü:** 3D backtracking çözücü (LevelSolver.cs) entegrasyonu, arama ağacının erken budanması ve çözüm yollarının sayılması.
- * **Faz 3 - Çözümünden Seviye Üretimi:** Parça seçici (Piece Selector), yerleşim kurucu (Solution Builder) ve başlangıç durumu karıştırıcısının (Scrambler) yazılması.
- * **Faz 4 - Zorluk Derecelendirmesi:** 3D seviye zorluk skoru hesaplama formülünün hayata geçirilmesi ve profil eşleşme kurallarının aktif edilmesi.
- * **Faz 5 - Editör Arayüz Entegrasyonu:** AI Level Designer editör penceresine görsel önizleme, çözüm oynatıcısı ve metrik dışı aktarma butonlarının eklenmesi.
- * **Faz 6 - Telemetri ve Kalibrasyon:** Oyuncu davranış verilerinin toplanması ve zorluk formüllerindeki katsayıların otomatik optimize edilmesi.

13. Hatalı Yaklaşımlar ve Koruma Kuralları

Seviye üretim sürecinde sıkça yapılan tasarım hataları ve bunların yerine uygulanması gereken doğru teknik yaklaşımlar aşağıda karşılaştırmalı olarak verilmiştir.

Hatalı Yaklaşım	Doğru Yaklaşım
Hedef şekil ve parçaları rastgele üretmek.	Önce çözümü kur; hedef şekli yerleşen parçalardan türet.
Zorluğu sadece parça sayısıyla ölçmek.	Dallanma, dikey katman geçişleri ve gecikmeli hataları ölç.
Sadece toplam parça hacmini eşitlemek.	Geometriyi ve dikey yerleşim sırasını çözücüyle doğrula.
Solver doğrulaması olmadan seviyeyi onaylamak.	Her seviye için oyundaki kurallarla çalışan çözücüü çalıştır.
Timeout/hata durumunda seviyeyi yayına almak.	Timeout durumunu "Belirsiz" kabul et ve seviyeyi reddet.
Parçaları sadece görsel prefab olarak saklamak.	3D hücre koordinatları ve geometrik özelliklerle sakla.

Zorunlu Koruma Kuralları (Sanity Rules):

- * **Kural 1:** Doğrulanmamış (validated == false) hiçbir seviye oyuna dahil edilemez.
- * **Kural 2:** Kütüphanede mükerrer (aynı imza) parça tespit edilirse uyarı verilmelidir.
- * **Kural 3:** Zorluk profilleri güncellendiğinde tüm kayıtlı seviyeler yeniden puanlanmalıdır.

Ek A. Varsayılan 3D Zorluk Profilleri

Parametre	Kolay	Orta	Zor	Uzman
Parça Adedi	3 - 5	5 - 8	7 - 11	9 - 14
Toplam Hacim (Küp)	12 - 24	20 - 40	32 - 60	45 - 80
Ortalama Parça Zorluğu	0 - 30	20 - 50	40 - 70	60 - 100
Dikey Katman Sayısı	1 - 2	2 - 3	3 - 4	4 - 5
Alternatif Çözüm Sayısı	3+	1 - 3	Mümkünse 1	Kesinlikle 1
Hedef Zorluk Skoru	0 - 24	25 - 49	50 - 74	75 - 100

Ek B. Seviye Kabul Kontrol Listesi

- ☐ Parçaların toplam hacmi (küp adedi) hedef şeklin hacmine eşit mi?
- ☐ Kayıtlı çözüm izi gerçek oyun kurallarıyla (katmanlar, engeller) tamamlanabiliyor mu?
- ☐ Bağımsız Solver seviye için en az bir geçerli çözüm buluyor mu?
- ☐ Benzersiz çözüm istenen seviyelerde alternatif ikinci bir çözümün olmadığı onaylandı mı?
- ☐ Seviyenin otomatik hesaplanan zorluk skoru (0-100) hedef profil sınırları içinde mi?
- ☐ Seed değeri kaydedildi ve aynı seed ile birebir aynı seviye tekrar üretilebiliyor mu?
- ☐ Mobil cihaz performans sınırları dahilinde doğrulama yapıldı mı?
- ☐ LevelData ScriptableObject dosyası "validated" olarak işaretlendi mi?

Önemli İlke: Önce çözümü kur, sonra oyuncunun başlangıç durumunu üret.